

PCM

Projet Final

2025-01-02

Loïc Frossard & Baptiste Dupertuis

HES-SO Master

2024-2025

Rapport

Table des matières

1 Introduction	3
1.1 Objectifs	3
2 Structure du programme	3
2.1 Schéma block	4
3 Analyse des résultats	5
3.1 Sans condition de fin sure (version 1)	5
3.1.1 Temps d'exécution	5
3.1.2 Speedup	6
3.1.3 Efficacité	6
3.2 Avec condition de fin sure (version 2)	6
3.2.1 Temps d'exécution	6
3.2.2 Speedup	6
3.2.3 Efficacité	6
3.3 Analyse des performances des deux versions	6
3.3.1 Variance sur le temps d'exécution des threads	6
3.3.2 Analyse de la concurrence sur la queue	7
3.3.3 Analyse de l'effet de MAX_DEPTH	7
4 Conclusion	7

1 Introduction

Ce rapport présente le projet final du cours de Programmation Concurrente et Multicœur (PCM). Le projet réalisé est capable de résoudre le Problème du voyageur de commerce (TSP, Traveling Salesman Problem) pour 19 villes en environ 13 minutes sur 10 threads.

1.1 Objectifs

L'objectif de ce projet est de développer un programme qui profite d'une architecture multicœur, en utilisant les techniques définies durant le cours de PCM.

Le travail consiste à résoudre le Problème du voyageur de commerce (TSP, Traveling Salesman Problem) avec une méthode branch-and-bound. Le programme doit visiter N villes une seule fois et doit revenir à la ville de départ, en faisant le chemin le plus court. Étant donné que le chemin est circulaire, le choix de la ville de départ n'a pas d'importance. Ceci est un problème difficile d'optimisation combinatoire, car s'il y a N villes alors, au départ d'une ville donnée, il y a $(N-1)!$ circuits différents qui passent par les $N-1$ autres villes et reviennent à la ville de départ. Il est supposé qu'il y a un chemin indépendant entre toute paire de villes.

2 Structure du programme

Le programme se base sur l'utilisation d'une méthode branch-and-bound pour résoudre le problème du voyageur de commerce. La méthode branch-and-bound est une technique de résolution de problèmes d'optimisation combinatoire. Elle consiste à diviser le problème en sous-problèmes plus petits, à évaluer ces sous-problèmes et à éliminer les branches qui ne peuvent pas contenir la solution optimale.

Pour ce faire, le programme utilise une approche parallèle, en divisant le problème en plusieurs sous-problèmes qui sont résolus en parallèle. Lorsqu'un problème est résolu, le programme vérifie si la solution trouvée est meilleure que la meilleure solution actuelle. Si c'est le cas, la meilleure solution est mise à jour. Les autres sous-problèmes sont ensuite évalués en fonction de la meilleure solution actuelle, afin d'éliminer les branches qui ne peuvent pas contenir la solution optimale. Ce processus est répété jusqu'à ce que tous les sous-problèmes soient résolus.

Le programme est écrit en C++ et utilise la bibliothèque `atomic` pour gérer les accès concurrents aux données partagées. Il utilise également la bibliothèque `thread` pour créer et gérer les threads. Le programme prend en entrée un fichier contenant les coordonnées des villes à visiter et affiche en sortie le chemin optimal ainsi que la distance totale parcourue.

La communication des problèmes entre les threads se fait à l'aide d'une queue. Cette queue n'utilise pas de lock pour garantir l'accès exclusif aux données partagées, mais utilise des opérations atomiques pour garantir la cohérence des données. L'implémentation de cette queue se base sur celle fournie dans le cours de PCM.

L'indication de la meilleure solution est faite à l'aide d'une variable partagée, cette variable est mise à jour de manière atomique pour garantir sa cohérence.

2.1 Schéma block

Le schéma suivant illustre le fonctionnement du programme. Ici la condition de fin pour un thread est la suivante : si la queue est vide, le thread s'arrête. Cette condition de fin n'est pas sûre, car il est possible que d'autres threads ajoutent des éléments à la queue après que le thread ait vérifié qu'elle était vide. Cela peut entraîner une situation où un thread s'arrête alors qu'il reste des éléments à traiter. Cependant, dans la pratique, cette situation ne s'est pas produite lors de nos tests. Nous avons aussi analysé un programme avec une condition de fin sûre et analysé l'impact sur les performances. La condition de fin sûre se base sur le comptage des chemins traités.

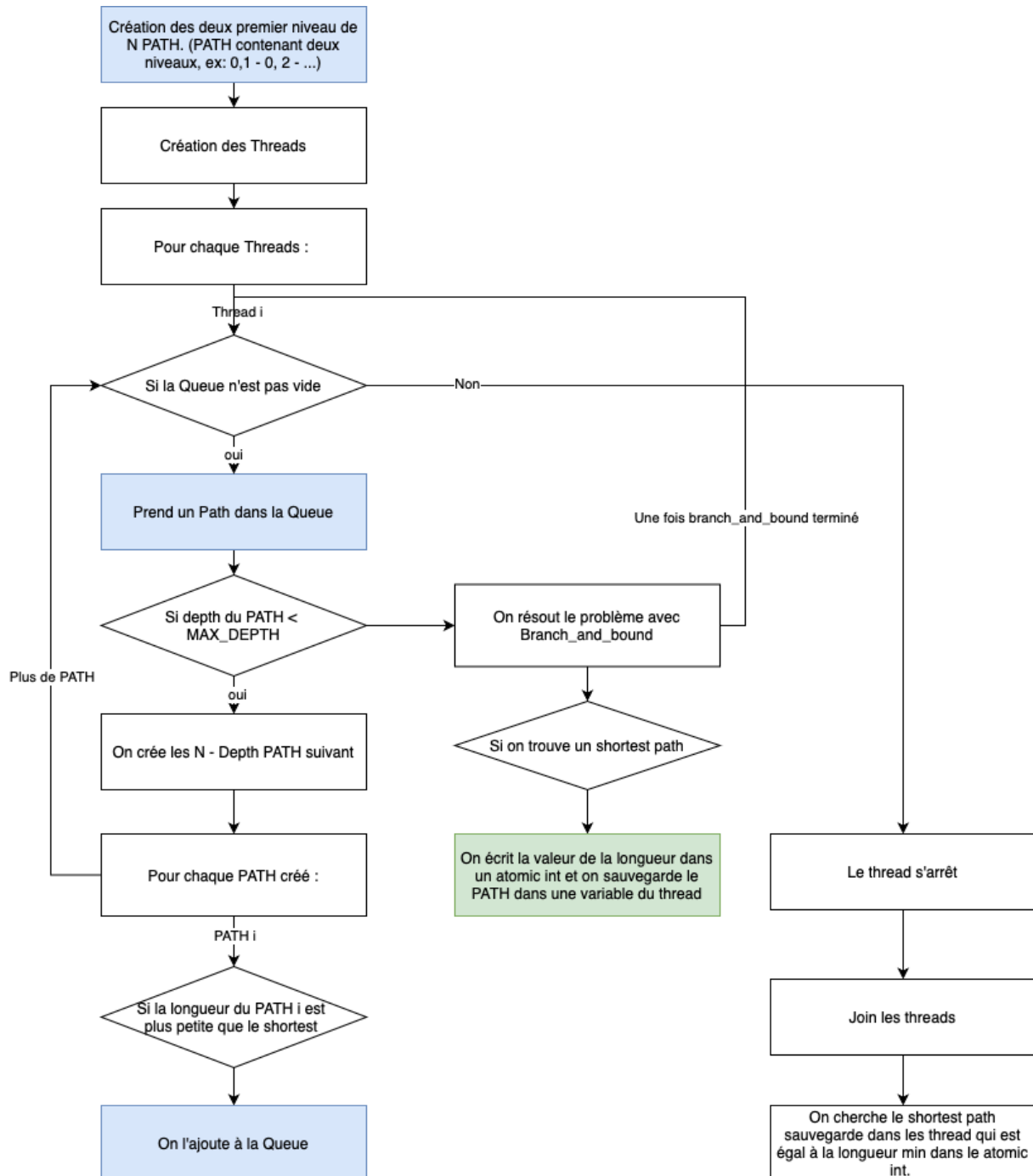


Fig. 1 — Schéma de fonctionnement du programme

Les cases bleues représente les actions qui interagissent avec la queue, les cases vertes représentent les actions qui interagissent avec la variable de la meilleure solution en écriture.

Voici les points clés du schéma :

- Le programme commence par initialiser la queue avec les deux ou trois premiers niveaux de l'arbre de recherche. Cela permet de créer assez de travail pour tous les threads et d'éviter un démarrage lent.
- Chaque thread récupère un sous-problème de la queue, si le sous-problème est suffisamment petit (il contient MAX_DEPTH villes), le thread le résout et met à jour la meilleure solution si il en trouve une. Si le problème est trop grand, le thread crée le prochain niveau de l'arbre de recherche et ajoute les sous-problèmes à la queue.
- Lorsqu'un thread a terminé de traiter un sous-problème, il en récupère un autre de la queue. Dans la version 1 du code, si la queue est vide, le thread s'arrête. Dans la version 2, le thread s'arrête uniquement si le nombre de sous-problèmes traités est égal au nombre total de sous-problèmes à traiter.

3 Analyse des résultats

Ce chapitre présente les résultats obtenus lors de l'exécution des deux versions du programme (avec et sans condition de fin sure). Les résultats sont basés sur des tests effectués sur un serveur de calcul avec 256 threads. Les tests ont été effectués pour un nombre de villes allant de 10 à 18.

3.1 Sans condition de fin sure (version 1)

3.1.1 Temps d'exécution

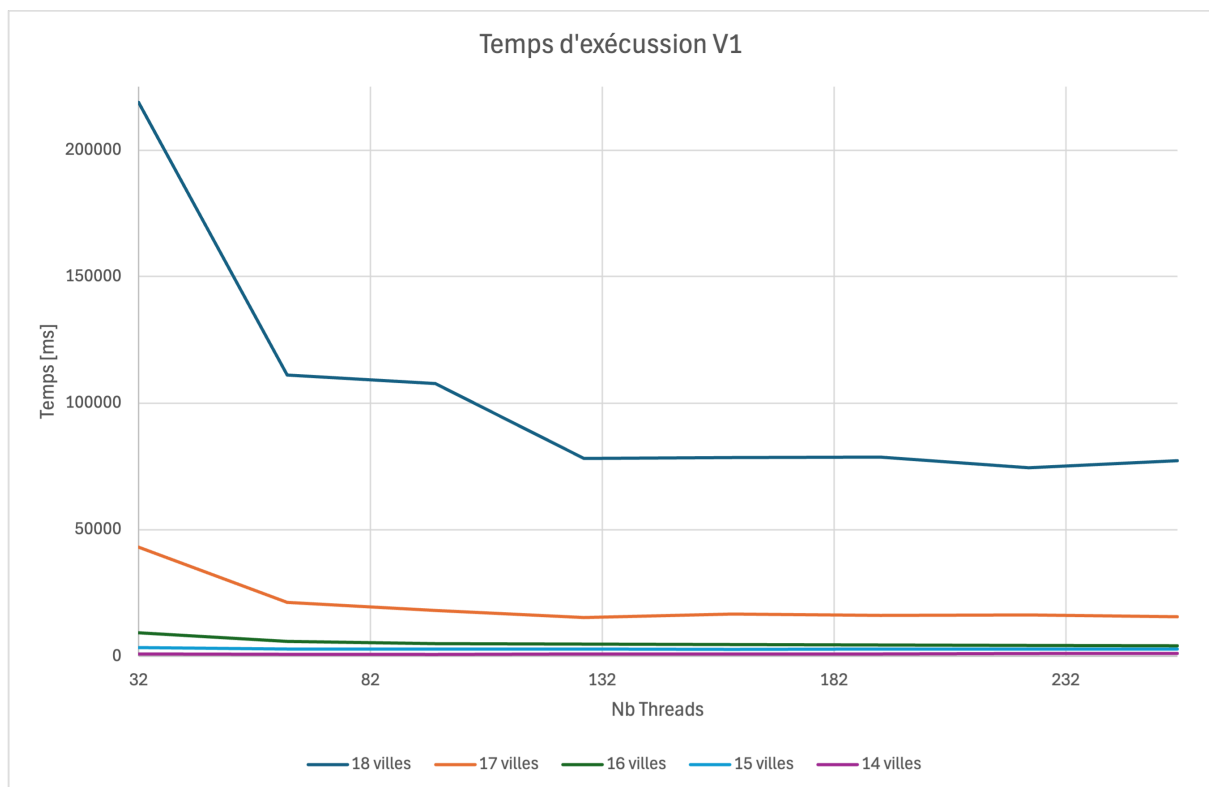


Fig. 2 — Temps d'exécution en fonction du nombre de villes - Version 1

3.1.2 Speedup

3.1.3 Efficacité

3.2 Avec condition de fin sure (version 2)

3.2.1 Temps d'exécution

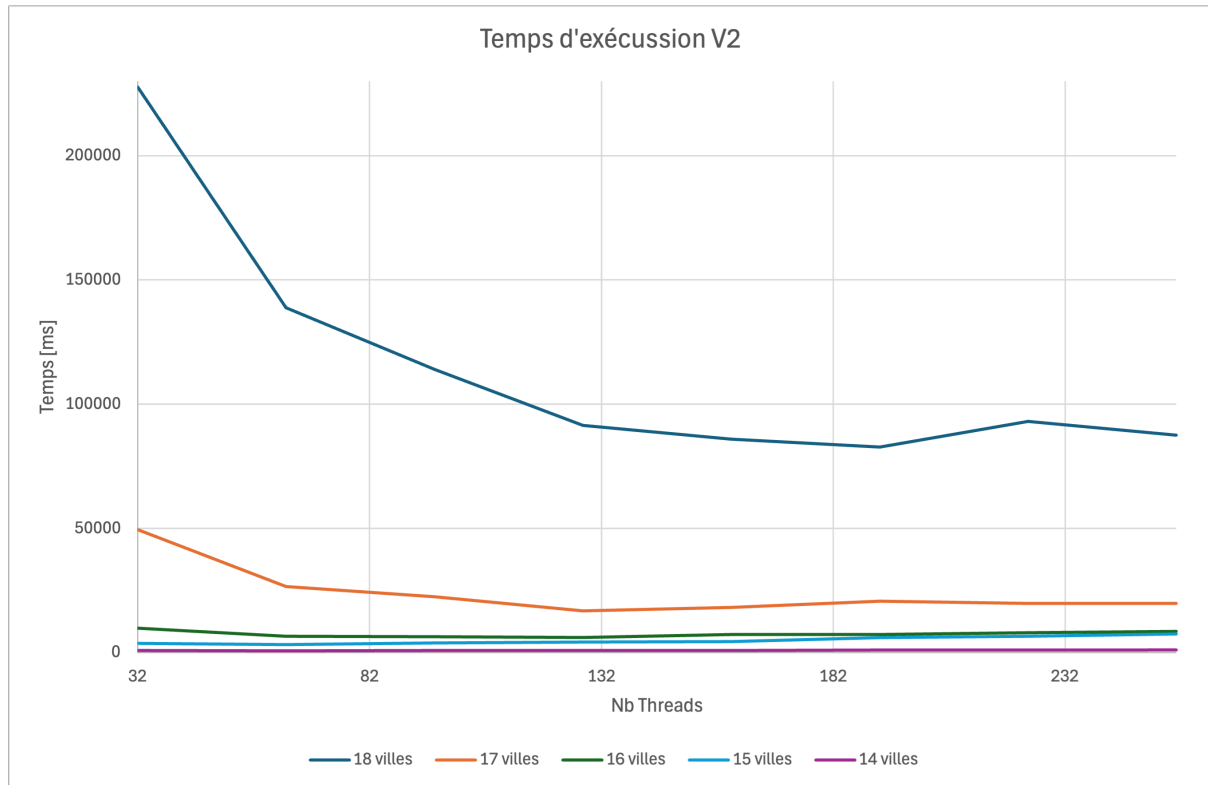


Fig. 3 — Temps d'exécution en fonction du nombre de villes - Version 2

3.2.2 Speedup

3.2.3 Efficacité

3.3 Analyse des performances des deux versions

3.3.1 Variance sur le temps d'exécution des threads

L'image suivant montre la variance sur le temps d'exécution des threads pour les deux versions du programme. On peut voir que la version 1 a une variance plus élevée que la version 2. Cela est dû au fait que dans la version 1, les threads s'arrêtent dès que la queue est vide, ce qui peut entraîner des différences de temps d'exécution entre les threads. Dans la version 2, les threads s'arrêtent uniquement lorsque le nombre de sous-problèmes traités est égal au nombre total de sous-problèmes à traiter, ce qui permet de réduire la variance sur le temps d'exécution des threads. Cependant, la variance reste relativement faible pour les deux versions, ce qui tend à montrer que les threads sont bien équilibrés en termes de charge de travail, même dans la version 1. Un grand pic est visible dans la version 1 aux alentours de 15 villes et qui descend pour les nombre de villes supérieurs. Ce pic est probablement dû au fait que pour 15 villes le temps d'exécution total est relativement court et donc la variance est plus visible.

car le problème est de taille modérée. Pour les villes inférieurs, le temps d'exécution est très (trop) cours, donc la variance est donc très faible et peu visible.

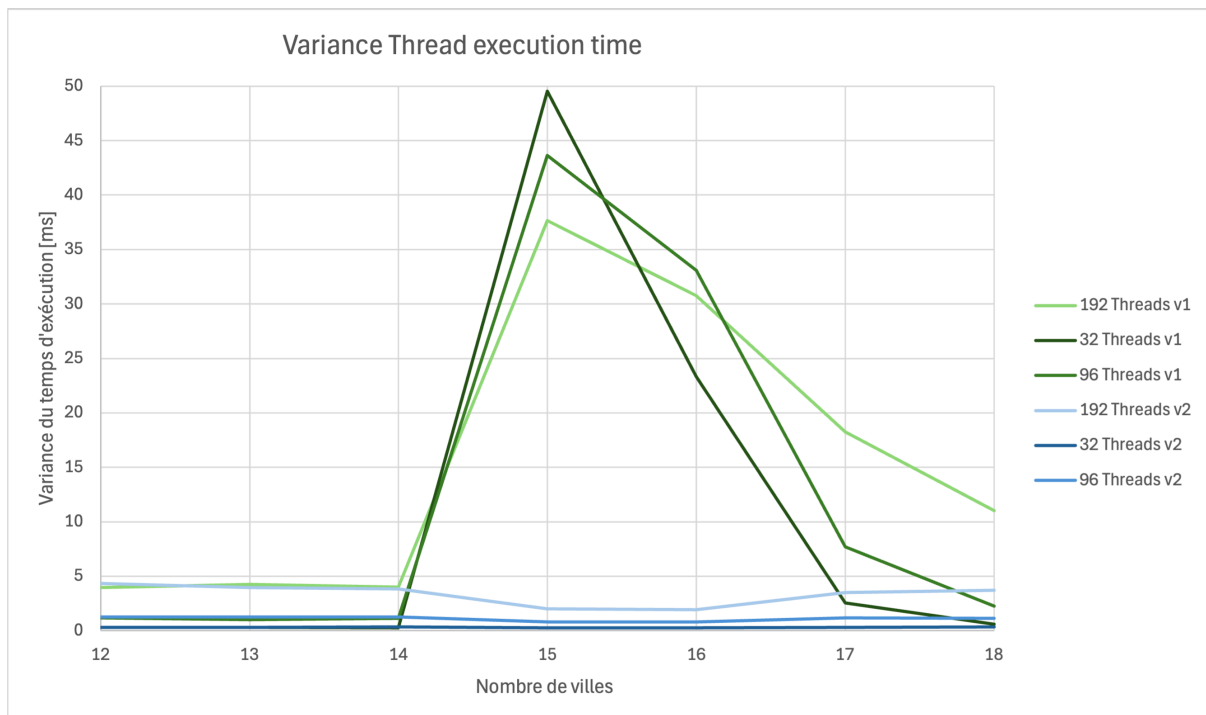


Fig. 4 — Variance sur le temps d'exécution des threads

3.3.2 Analyse de la concurrence sur la queue

3.3.3 Analyse de l'effet de MAX_DEPTH

4 Conclusion